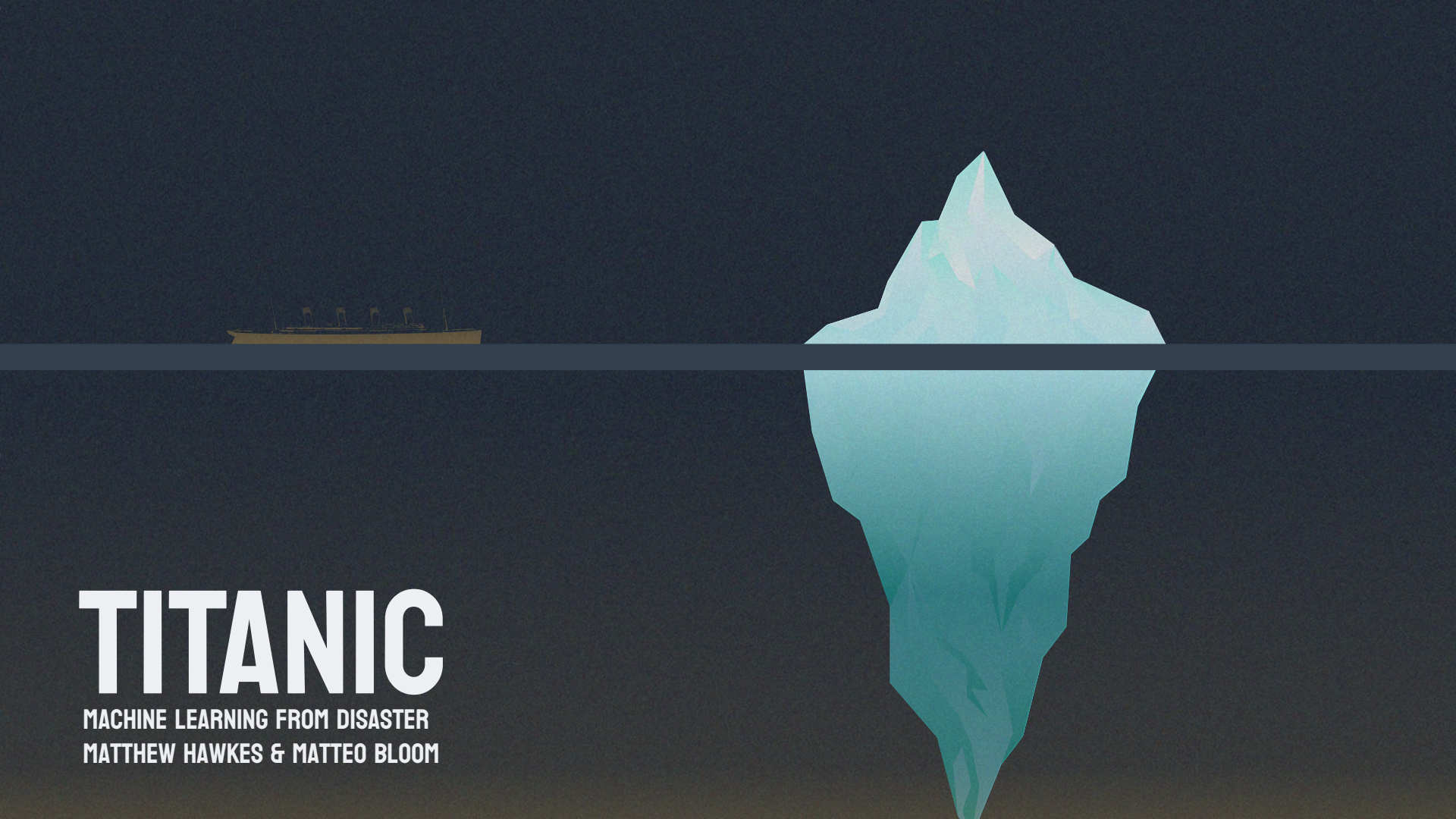


TITANIC

MACHINE LEARNING FROM DISASTER

MATTHEW HAWKES & MATTEO BLOOM

PROJECT OBJECTIVE

Objective:

- Build a predictive model to estimate the probability a passenger survived the Titanic disaster.

Success Metric:

- Achieve strong performance on the Kaggle leaderboard.
- Produce a clean, reproducible ML notebook.
- Write comprehensive comments explaining our process and results.

PROJECT WORKFLOW

PART 1 — **IMPORT LIBRARIES & DATASETS** - Import necessary machine learning libraries and the titanic dataset

PART 2 — **EXPLORATORY DATA ANALYSIS** - Visualize correlation and trends between variables in the dataset

PART 3 — **FEATURE ENGINEERING** - Create useful features to use for our model training and prediction

PART 4 — **PREPROCESSING** - Prepare a preprocessor column transformer to standardize numerical and categorical data

PART 5 — **DETERMINING THE BEST MODEL** - Create an initial pipeline that compares models to determine the best

PART 6 — **HYPERPARAMETER TUNING & PREDICTION** - Tune the random forests hyperparameters and predict on the test data.

EXPLORATORY DATA ANALYSIS

Why EDA?

- Identify dominant survival patterns
- Surface biases, missingness, and non-linear effects
- Guide feature engineering and model selection

Sex:

- Women survival: ~74%
- Men survival: ~19%
  - Sex is the strongest single predictor.

Class:

- 1st Class: 63% survival
- 3rd Class: 24% survival
  - Socioeconomic status strongly affects outcome.

Age:

- Children survive more often
  - Non-linear trend - motivates binning.

Family Size:

- Best survival with medium size families (2–3)
  - Large families and solo travelers do worse.

EXPLORATORY DATA ANALYSIS/FEATURE ENGINEERING

The EDA informed feature engineering choices:

- Create FamilySize and IsAlone
- Bin Age and Fare
- Encode Sex cleanly since it's highly predictive
- Anticipate non-linear model performance (trees > linear)

```
============================================================
TITANIC EXPLORATORY DATA ANALYSIS
============================================================

Train shape: (891, 12) | Test shape: (418, 11)
Survival rate: 38.38%

Missing Values:
Age         177
Cabin       687
Embarked      2
dtype: int64
```

```
============================================================
KEY STATISTICS
============================================================

Survival by Sex:
            mean  count
Sex
female  0.742038    314
male    0.188908    577

Survival by Pclass:
            mean  count
Pclass
1       0.629630    216
2       0.472826    184
3       0.242363    491

Survival by Embarked:
              mean  count
Embarked
C         0.553571    168
Q         0.389610     77
S         0.336957    644

Key Insights:
• Women had 74% survival vs 19% for men
• 1st class: 63% survival, 3rd class: 24% survival
• Children had higher survival rates
• Small families (2-3) had better survival than large families or solo travelers
```

FEATURE ENGINEERING

Goals:

1. Extract meaningful structure EDA revealed
2. Reduce noise from raw numeric values
3. Prepare categorical variables for ML models

Engineered Features:

- FamilySize = SibSp + Parch + 1
- IsAlone = 1 if FamilySize == 1 else 0
- AgeBin (child, teen, young adult, adult, senior)
- FareBin (quartiles)
- Encoded Sex and Embarked

```python
train_df['Age'] = train_df['Age'].fillna(train_df['Age'].median())
test_df['Age'] = test_df['Age'].fillna(test_df['Age'].median())

train_df['Embarked'] = train_df['Embarked'].fillna(train_df['Embarked'].mode()[0])
test_df['Embarked'] = test_df['Embarked'].fillna(test_df['Embarked'].mode()[0])

test_df['Fare'] = test_df['Fare'].fillna(test_df['Fare'].median())

train_df['FamilySize'] = train_df['SibSp'] + train_df['Parch'] + 1
test_df['FamilySize'] = test_df['SibSp'] + test_df['Parch'] + 1

train_df['IsAlone'] = 1
train_df.loc[train_df['FamilySize'] > 1, 'IsAlone'] = 0

test_df['IsAlone'] = 1
test_df.loc[test_df['FamilySize'] > 1, 'IsAlone'] = 0

train_df['Sex'] = train_df['Sex'].map({'male': 0, 'female': 1})
test_df['Sex'] = test_df['Sex'].map({'male': 0, 'female': 1})

train_df['Embarked'] = train_df['Embarked'].map({'S':0, 'C':1, 'Q':2})
test_df['Embarked'] = test_df['Embarked'].map({'S':0, 'C':1, 'Q':2})

train_df['AgeBin'] = pd.cut(train_df['Age'], bins=[0, 12, 18, 35, 60, 80], labels=False)
test_df['AgeBin'] = pd.cut(test_df['Age'], bins=[0, 12, 18, 35, 60, 80], labels=False)

train_df['FareBin'] = pd.qcut(train_df['Fare'], 4, labels=False)
test_df['FareBin'] = pd.qcut(test_df['Fare'], 4, labels=False)
```

Key Impact:
Improved tree-based model performance by capturing interactions the raw data hid.

PREPROCESSING

- Dropped irrelevant columns that don't help prediction: PassengerId, Name, Ticket, Cabin.
- Separated target variable (Survived) from the feature set.

- **Numeric preprocessing pipeline:**
  - Filled missing values using median imputation.
  - Applied StandardScaler to normalize numeric features.

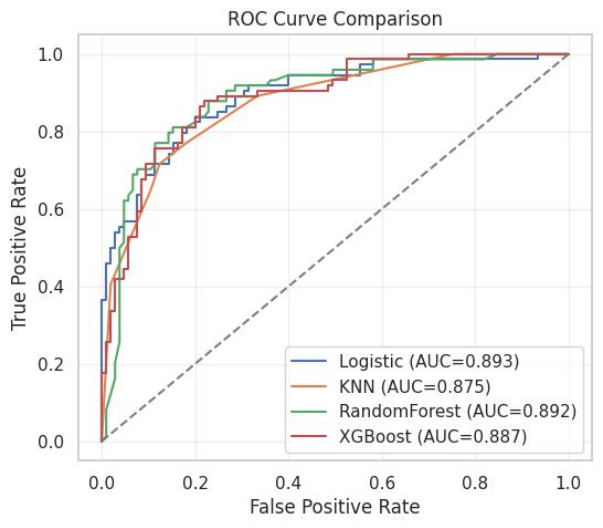
- **Categorical preprocessing pipeline:**
  - Filled missing values using the most frequent category.
  - Converted categories to numbers using OneHotEncoder.

- Combined both pipelines into a single ColumnTransformer (preprocessor).

```python
# Drop columns we don't want to model on right now
cols_to_drop = ['PassengerId', 'Name', 'Ticket', 'Cabin']
train_df_model = train_df.drop(columns=cols_to_drop + ['Survived'])
test_df_model  = test_df.drop(columns=cols_to_drop)

y = train_df['Survived']  # target

# Identify numeric and categorical columns
numeric_features = train_df_model.select_dtypes(include=['int64', 'float64']).columns
categorical_features = train_df_model.select_dtypes(include=['object']).columns

# Pipelines for numeric + categorical preprocessing
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Combine into a single preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)
```

MODELING

- Determined the best model by comparing metrics between the 4 models.
- Found that Random Forest performed the best overall.
- Highest overall accuracy, precision (0.827)
- Very strong F1-Score and recall (0.786)
- Handles non-linear relationships
- Less sensitive to overfitting



	Model	Accuracy	Precision	Recall	F1-score
0	Logistic	0.798883	0.779412	0.716216	0.746479
1	KNN	0.810056	0.803030	0.716216	0.757143
2	RandomForest	0.826816	0.802817	0.770270	0.786207
3	XGBoost	0.821229	0.769231	0.810811	0.789474

```
=== Logistic ===
[[90 15]
 [21 53]]
              precision    recall  f1-score   support

Not Survived       0.81      0.86      0.83       105
    Survived       0.78      0.72      0.75        74

    accuracy                           0.80       179
   macro avg       0.80      0.79      0.79       179
weighted avg       0.80      0.80      0.80       179
```

```
=== KNN ===
[[92 13]
 [21 53]]
              precision    recall  f1-score   support

Not Survived       0.81      0.88      0.84       105
    Survived       0.80      0.72      0.76        74

    accuracy                           0.81       179
   macro avg       0.81      0.80      0.80       179
weighted avg       0.81      0.81      0.81       179
```

```
=== RandomForest ===
[[91 14]
 [17 57]]
              precision    recall  f1-score   support

Not Survived       0.84      0.87      0.85       105
    Survived       0.80      0.77      0.79        74

    accuracy                           0.83       179
   macro avg       0.82      0.82      0.82       179
weighted avg       0.83      0.83      0.83       179
```

```
=== XGBoost ===
[[87 18]
 [14 60]]
              precision    recall  f1-score   support

Not Survived       0.86      0.83      0.84       105
    Survived       0.77      0.81      0.79        74

    accuracy                           0.82       179
   macro avg       0.82      0.82      0.82       179
weighted avg       0.82      0.82      0.82       179
```

HYPERPARAMETER TUNING

- Tune the random forest's hyperparameters, after determining it's the best model.
- Created a parameter dictionary for the hyperparameter search space:
  - Number of trees
  - Tree depth
  - Minimum split size
  - Minimum leaf size
  - Feature selection strategy
- Used KFold Cross Validation and performed a RandomizedSearchCV, optimizing hyperparameters.
- Created our tuned RandomForest model that was ready for prediction.

```python
# Wrap RF with preprocessing so it works on the raw Titanic data
rf_pipe = Pipeline([
    ('prep', preprocessor),   # ColumnTransformer from before
    ('model', rf)
])

# Hyperparameter search space
param_dist = {
    'model__n_estimators': [200, 400, 600, 800],
    'model__max_depth': [None, 10, 20, 30, 40],
    'model__min_samples_split': [2, 5, 10],
    'model__min_samples_leaf': [1, 2, 4],
    'model__max_features': ['auto', 'sqrt', 'log2']
}

# Cross-validation
cv = KFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE)

# RandomizedSearchCV
rand_search = RandomizedSearchCV(
    estimator=rf_pipe,
    param_distributions=param_dist,
    n_iter=20,
    scoring='accuracy',
    cv=cv,
    n_jobs=-1,
    random_state=RANDOM_STATE,
    verbose=1
)

# Fit model
rand_search.fit(X_train, y_train)
```

SUBMISSION & EVALUATION

- Printed out the best parameters and CV accuracy
- Predicted off the test dataset and created a submission file
- Compiled all analysis: cleaned dataset, Python notebooks, visualizations
- Polished code, visuals, and comments.
- Submitted the completed project for review

```python
# Fit model
rand_search.fit(X_train, y_train)

# Results
print("Best parameters:", rand_search.best_params_)
print("Best CV Accuracy:", rand_search.best_score_)

# Retrain best RF on all data
best_model = rand_search.best_estimator_
best_model.fit(train_df_model, y)

# Predict
kaggle_preds = best_model.predict(test_df_model)

# Create submission file
submission = pd.DataFrame({
    'PassengerId': test_df['PassengerId'].values,
    'Survived': kaggle_preds
})
submission.to_csv('submission.csv', index=False)
submission.head()
```

FINAL REFLECTION

- **EDA discoveries & insights**
  - Women survival rate: ~74% vs. men ~19% - I found this magnitude of difference surprising
  - First-class survival: ~63% vs. third-class ~24% - Revealed how social/economic factors shaped outcomes
  - Historically interesting and practically useful for feature engineering
- **Preprocessing & feature engineering choices**
  - Created FamilySize and IsAlone - I noticed these had the biggest impact on the feature importance list.
  - Binned Age and Fare - Captured non-linear relationships that other variables missed
  - I observed small families (2–3) had higher survival rates than solo travelers or very large families

FINAL REFLECTION

- **Modeling lessons learned**
  - Logistic Regression: solid baseline
  - KNN: struggled with scaling - I realized scaling issues limited performance
  - Tree-based models (Random Forest, XGBoost) consistently best
  - I learned Random Forest offered robustness and interpretability, while XGBoost gave slightly higher scores but required careful tuning
  - Random Forest chosen - I liked its balance of performance, simplicity, and stability

- **Future improvements (if more time)**
  - Explore ensemble methods (stacking Logistic Regression, Random Forest, XGBoost)
  - Perform feature importance analysis to refine features
  - Test advanced imputation techniques (iterative/model-based)
  - Incorporate external data (e.g., passenger titles from names) - I believe this could improve our prediction further

THANK YOU